

EDUPAIE

Application de gestion des paiements scolaires

Projet individuel — Python / PySide6 / SQLite

1. Présentation du projet

EduPaie est une application desktop destinée à faciliter la gestion des frais de scolarité dans un établissement scolaire.

L'application répond à un problème fréquent : le suivi manuel des paiements dans des cahiers ou des fichiers Excel rend difficile la connaissance du montant réellement payé par chaque élève et du montant restant à encaisser.

EduPaie permet donc de centraliser les informations relatives aux élèves et à leurs paiements, de calculer automatiquement les soldes et de générer des reçus fiables et numérotés.

2. Fonctionnalités réalisées

L'application propose les fonctionnalités suivantes :

- ajout d'un élève ;
- modification des informations d'un élève ;
- suppression d'un élève ;
- recherche et filtrage des élèves ;
- enregistrement d'un paiement ;
- choix du mode de paiement : espèces, chèque, virement ou Mobile Money ;
- contrôle du montant payé par rapport au solde restant ;
- calcul automatique du solde ;
- détermination du statut de paiement ;
- consultation de l'historique des paiements ;
- génération de reçus PDF numérotés ;
- consultation des reçus déjà générés ;
- tableau de bord avec les principaux indicateurs.

Les statuts utilisés sont :

- **Non payé** : aucun paiement enregistré ;

- **Partiellement payé** : une partie des frais a été payée ;
- **Soldé** : la totalité des frais a été payée.

3. Architecture de l'application

EduPaie respecte une architecture en couches.

Couche d'accès aux données

Cette couche est constituée notamment du dossier `repositories/`.

Les repositories encapsulent les requêtes SQL et assurent les opérations sur la base SQLite.

Exemples :

- `eleve_repository.py`
- `paiement_repository.py`

Le dossier `database/` contient la connexion à SQLite et le schéma de la base.

Couche métier

La logique métier se trouve dans le dossier `services/`.

Elle prend notamment en charge :

- les calculs de soldes ;
- les validations des paiements ;
- les statuts de paiement ;
- la génération des numéros de reçus ;
- les données du tableau de bord.

Exemples :

- `eleve_service.py`
- `paiement_service.py`
- `recu_service.py`
- `dashboard_service.py`

Couche interface

L'interface graphique est développée exclusivement avec PySide6.

Elle se trouve dans le dossier ui/.

Elle contient notamment :

- la fenêtre principale ;
- la gestion des élèves ;
- l'enregistrement des paiements ;
- l'historique ;
- les reçus ;
- le tableau de bord.

Cette séparation évite qu'un widget de l'interface exécute directement une requête SQL.

4. Modélisation des données

La base de données utilise SQLite.

Elle contient principalement deux tables : *eleves* et *paiements*.

*Table **eleves***

- *id* : clé primaire ;
- *nom* ;
- *prenom* ;
- *classe* ;
- *annee_scolaire* ;
- *montant_total*.

*Table **paiements***

- *id* : clé primaire ;
- *numero_recu* : numéro unique du reçu ;
- *eleve_id* : clé étrangère vers *eleves* ;
- *montant* ;
- *date_paiement* ;
- *mode_paiement*.

La relation entre les deux tables est :

ELEVE (1,N) PAIEMENT

Un élève peut donc avoir plusieurs paiements tandis qu'un paiement appartient à un seul élève.

5. Règles métier

Le solde d'un élève est calculé selon la formule :

Solde = montant total dû – somme des paiements

L'application empêche également l'enregistrement d'un paiement supérieur au solde restant.

Les montants doivent être positifs et les champs obligatoires doivent être renseignés.

Chaque paiement possède un numéro de reçu unique.

6. Jeu de données de démonstration

La base SQLite livrée avec le projet contient **15 élèves**.

Le jeu de données permet de démontrer les différents états de paiement :

- **6 élèves soldés ;**
- **6 élèves partiellement payés ;**
- **3 élèves non payés.**

Cette répartition permet de tester les calculs de soldes, les statuts et le tableau de bord.

7. Tests

Le projet dispose de **21 tests automatisés**.

Les tests couvrent notamment :

- les repositories ;
- les services ;
- les paiements ;
- les calculs ;
- les reçus ;
- le tableau de bord.

L'ensemble des tests a été exécuté avec succès.

8. Génération des reçus

Après l'enregistrement d'un paiement, EduPaie génère un reçu contenant notamment :

- le numéro du reçu ;
- les informations de l'élève ;
- le montant payé ;
- la date ;
- le mode de paiement ;
- le solde restant après paiement.

Le reçu peut ensuite être consulté depuis l'historique.

9. Packaging

Une version Windows autonome a été générée avec PyInstaller.

L'exécutable `EduPaie.exe` permet d'utiliser l'application sur un ordinateur Windows sans installer Python ni les dépendances du projet.

L'exécutable a été généré puis testé afin de vérifier le fonctionnement de l'application.

10. Choix techniques

Python a été choisi pour sa simplicité et son écosystème.

PySide6 a été utilisé conformément aux contraintes du brief pour construire l'interface graphique.

SQLite permet de disposer d'une base relationnelle légère, locale et ne nécessitant pas de serveur.

L'architecture Repository / Service / UI permet de séparer clairement l'accès aux données, la logique métier et l'interface.

11. Limites connues

La version actuelle est destinée principalement à une utilisation locale sur un poste.

Elle ne propose pas encore :

- de synchronisation entre plusieurs postes ;
- de gestion multi-utilisateur ;

- de serveur centralisé ;
- de gestion de plusieurs établissements.

Ces fonctionnalités pourraient être ajoutées dans une version future.

12. Conclusion

EduPaie répond aux besoins définis dans le brief en proposant une solution complète pour gérer les élèves, les paiements, les soldes, l'historique et les reçus.

Le projet respecte les contraintes techniques principales : Python, PySide6, SQLite, architecture en couches, tests automatisés, gestion Git et génération d'un exécutable Windows.